PARCOURS PERSONNALISÉ · HTML · CSS · JAVASCRIPT VANILLA

LifeBoard

10 projets de consolidation, un seul fil rouge

Un tableau de bord personnel qui s'enrichit à chaque étape,
de la simple page d'accueil jusqu'à l'application publiée en ligne.

1 2 3 4 5 6 7 8 9 10

10	1	46–61 h	0
étapes progressives	projet fil rouge	de pratique estimées	framework, 0 build tool

LES QUATRE PHASES

- **Fondations** — étapes 1 à 3
- **Structure & données** — étapes 4 à 6
- **Applications riches** — étapes 7 à 9
- **Finalisation** — étape 10

APPRENANTE	MENTOR	DÉMARRAGE
Madefo Fortune	**Louis**	**Octobre 2026**

Prérequis : bases de HTML, CSS et JavaScript (variables, conditions, boucles, fonctions, tableaux, objets, DOM, événements, fetch basique, localStorage).

Le fil rouge : LifeBoard

Ce parcours a été conçu par Louis, mentor, pour **Madefo Fortune**. Plutôt que dix exercices isolés, la progression s'appuie sur **un seul projet** : un tableau de bord personnel appelé LifeBoard qui centralise tâches, budget, météo, notes, habitudes et statistiques. Chaque étape ajoute **un module autonome et utilisable seul** ; à partir de l'étape 5, les modules s'assemblent dans une vraie application.

Pourquoi cette approche ?

- **Motivation** : Madefo construit un outil qu'elle peut réellement utiliser au quotidien et montrer (portfolio).
- **Continuité** : les acquis sont réutilisés d'étape en étape (tâches → budget → statistiques) sans repartir de zéro.
- **Réalisme** : elle vit un vrai cycle de projet, avec refactorisation, intégration et livraison.
- **Souplesse** : chaque étape reste validable indépendamment si le rythme ralentit.

Vue d'ensemble de la progression

N°	Module	Notion nouvelle principale	Durée	Niveau
1	**Accueil vivant**	`Date`, `setInterval`, `Math.random`	2 à 3 h	● ○ ○ ○ ○
2	**Liste de tâches**	`createElement`, état → `render()`	3 à 4 h	● ○ ○ ○ ○
3	**Tâches persistantes**	`localStorage`, JSON, `filter`	3 à 4 h	● ● ○ ○ ○
4	**Gestionnaire de budget**	Validation, `map` / `reduce`	4 à 5 h	● ● ○ ○ ○
5	**Squelette de l'application**	Modules ES, routage par hash	4 à 6 h	● ● ● ○ ○
6	**Station météo**	`fetch`, `async` / `await`	5 à 6 h	● ● ● ○ ○
7	**Carnet de notes**	CRUD, debounce, `<dialog>`	5 à 7 h	● ● ● ○ ○
8	**Suivi d'habitudes**	Algorithmes de dates, tests	6 à 8 h	● ● ● ● ○
9	**Tableau de statistiques**	Canvas 2D, agrégation	6 à 8 h	● ● ● ● ○
10	**Mise en production**	Thèmes, import/export, a11y	8 à 10 h	● ● ● ● ●

Total estimé pour Madefo : **46 à 61 heures**, soit environ 8 à 10 semaines à raison de 6 heures par semaine. Les durées incluent les blocages normaux et les phases de débogage d'une débutante.

Méthode d'accompagnement

Un cycle simple et répétable pour chaque étape :

1	**Cadrer**	Lire ensemble le cahier des charges (10 min). Madefo reformule avec ses mots ce qu'elle va construire.
2	**Planifier**	Maquette papier + découpage en petites tâches. Une fonctionnalité = une tâche.
3	**Coder par incréments**	Tester dans le navigateur après chaque fonctionnalité, jamais après trois heures de code.
4	**Relire**	Revue de code avec le mentor : lisibilité, nommage, cas limites.
5	**Rétrospective**	Trois questions : qu'est-ce qui a marché ? Qu'est-ce qui a bloqué ? Qu'ai-je appris ?

Règles d'or du mentor

- **Questionner plutôt que corriger** : « Qu'attends-tu ? Qu'obtiens-tu ? Où le vois-tu dans la console ? »
- **Règle des 20 minutes** : avant de demander de l'aide, 20 minutes de recherche (MDN, console, relecture du code).
- **Ne jamais donner la solution complète** : un indice, puis un deuxième, puis un exemple sur un cas voisin.
- **Git recommandé dès l'étape 1** : un commit par fonctionnalité, avec un message clair. L'historique devient un journal de progression.
- **Définition de « terminé »** : cahier des charges respecté, zéro erreur en console, code relu, contraintes techniques vérifiées.

Architecture cible du projet final

```
lifeboard/
├── index.html
├── README.md
├── css/
│   ├── base.css          variables, reset, thèmes
│   ├── layout.css        grille, barre latérale
│   └── modules.css       styles propres à chaque module
└── js/
    ├── main.js           point d'entrée
    ├── router.js         routage par hash (étape 5)
    ├── storage.js        accès localStorage, import/export
    ├── modules/
    │   ├── accueil.js    taches.js    budget.js
    │   └── meteo.js      notes.js     habitudes.js    stats.js
    └── utils/
        ├── dates.js      format.js    charts.js
```

Pendant les étapes 1 à 4, une structure plus simple (un fichier JS par étape) est tout à fait acceptable. La réorganisation en modules constitue justement l'objectif de l'étape 5.

1

PHASE A · FONDATIONS

Accueil vivant

Salutation, horloge et citation du jour

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
2 à 3 h	●○○○○	Objet `Date`, `setInterval`, `Math.random`, gabarits de chaînes	Page d'accueil du tableau de bord

OBJECTIF PÉDAGOGIQUE

Consolider la manipulation du DOM et des tableaux d'objets, tout en découvrant la gestion du temps en JavaScript. Mise en place de la structure de dossiers (`index.html`, `css/`, `js/`) qui servira pendant tout le parcours.

DESCRIPTION FONCTIONNELLE

Une page d'accueil qui s'adresse à l'utilisatrice par son prénom, affiche l'heure en direct et propose une citation inspirante. C'est la porte d'entrée de LifeBoard : simple, mais déjà vivante.

FONCTIONNALITÉS CLÉS

- Salutation adaptée à l'heure (« Bonjour », « Bon après-midi », « Bonsoir ») avec le prénom saisi dans un champ
- Horloge mise à jour chaque seconde et date complète en français (ex. « lundi 12 octobre 2026 »)
- Citation aléatoire tirée d'un tableau d'objets `{texte, auteur}` ; le bouton « Nouvelle citation » ne répète pas la précédente
- Compte à rebours en jours jusqu'à une date importante (examen, anniversaire, objectif)

CONTRAINTES TECHNIQUES

- Pas de librairie de dates (moment.js, dayjs)
- `textContent` plutôt que `innerHTML`
- Au moins 10 citations dans le tableau
- Une fonction = une responsabilité
- HTML sémantique (`header`, `main`, `section`)

PISTES DE BONUS

- Citation stable pour toute la journée (calculée à partir du jour de l'année)
- Couleur de fond qui change selon la période (matin, soir, nuit)
- Bouton « Copier la citation » (`navigator.clipboard`)

COMPÉTENCES TRANSVERSES

Lecture de la documentation MDN (`Date`, `toLocaleDateString`), usage de la console pour déboguer, nommage clair des variables, organisation des fichiers.

2

PHASE A · FONDATIONS

Liste de tâches

Le réflexe « état → affichage »

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
3 à 4 h	●○○○○	`createElement`, motif état → `render()`, `classList`, `preventDefault`	Module Tâches (version en mémoire)

OBJECTIF PÉDAGOGIQUE

Comprendre le principe de toute interface dynamique : les données vivent dans une variable (l'état) et une fonction `render()` les affiche. Cette habitude évite la majorité des bugs d'affichage.

DESCRIPTION FONCTIONNELLE

Une liste de tâches classique : l'utilisatrice ajoute, coche et supprime des tâches. Les données disparaissent au rechargement de la page (c'est volontaire : ce sera l'objet de l'étape 3).

FONCTIONNALITÉS CLÉS

- Ajout d'une tâche via un formulaire (touche Entrée incluse), refus des textes vides ou composés d'espaces
- Cocher / décocher une tâche (style barré via une classe CSS)
- Suppression d'une tâche avec un bouton dédié
- Compteur « X tâches restantes » et message d'état vide (« Rien à faire, profitez ! »)
- Chaque tâche est un objet `{id, titre, terminee}` stocké dans un tableau

CONTRAINTES TECHNIQUES

- Le tableau `taches` est la seule source de vérité
- `render()` reconstruit la liste à chaque changement
- Pas de `localStorage` (étape 3)
- Aucun attribut `onclick` dans le HTML : `addEventListener` uniquement

PISTES DE BONUS

- Modifier le titre d'une tâche par double-clic
- Bouton « Supprimer les tâches terminées »
- Niveau de priorité (haute, moyenne, basse) avec code couleur

COMPÉTENCES TRANSVERSES

Raisonnement par état, débogage pas à pas avec les points d'arrêt des DevTools, gestion des cas limites (liste vide, saisie vide), découpage logique.

3

PHASE A · FONDATIONS

Tâches persistantes

Sauvegarde, filtres et échéances

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
3 à 4 h	●●○○○	`localStorage`, `JSON.stringify` / `parse`, `try…catch`, délégation d'événements, `filter`	Module Tâches (version complète)

OBJECTIF PÉDAGOGIQUE

Faire survivre les données au rechargement de la page et manipuler des tableaux avec `filter`. Premier contact avec la gestion d'erreurs (données stockées corrompues).

DESCRIPTION FONCTIONNELLE

La liste de l'étape 2 devient un vrai outil du quotidien : les tâches sont conservées, classées par catégorie, filtrables et munies d'une date d'échéance. Les retards sont signalés visuellement.

FONCTIONNALITÉS CLÉS

- Sauvegarde automatique après chaque modification et rechargement de l'état au démarrage
- Filtres « Toutes / À faire / Terminées »
- Catégories (Études, Perso, Projets…) choisies dans une liste déroulante
- Date d'échéance avec mise en évidence des tâches en retard
- Un seul écouteur sur la liste (délégation via `event.target.closest()`)

CONTRAINTES TECHNIQUES

- Deux fonctions dédiées : `chargerEtat()` et `sauvegarderEtat()`
- `try…catch` si le JSON stocké est invalide
- Clé préfixée (`lifeboard:taches`)
- `filter()` obligatoire (pas de boucle `for` + `if`)

PISTES DE BONUS

- Tri par date d'échéance
- Bouton « Annuler » après une suppression (undo)
- Pourcentage de tâches terminées avec barre de progression

COMPÉTENCES TRANSVERSES

Lecture de l'onglet « Application » des DevTools, anticipation des erreurs, refactorisation du code de l'étape 2 sans casser l'existant.

4

PHASE B · STRUCTURE & DONNÉES

Gestionnaire de budget

Revenus, dépenses et solde en FCFA

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
4 à 5 h	●●○○○	Formulaires multi-champs, validation personnalisée, `map` / `reduce`, `Intl.NumberFormat`	Module Budget

OBJECTIF PÉDAGOGIQUE

Travailler les formulaires riches, la validation côté client et les méthodes de tableaux (`map`, `filter`, `reduce`) pour calculer des totaux. Premier module avec une vraie logique métier.

DESCRIPTION FONCTIONNELLE

Un suivi de budget personnel : chaque mouvement d'argent est enregistré avec son montant, sa catégorie et sa date. L'application calcule les totaux et le solde, avec une présentation adaptée au franc CFA.

FONCTIONNALITÉS CLÉS

- Formulaire : type (revenu ou dépense via boutons radio), montant, catégorie, date, description
- Validation avec messages d'erreur sous les champs (montant > 0, date valide…)
- Liste des opérations avec suppression et filtres par mois et par type
- Totaux revenus / dépenses / solde calculés avec `reduce`, solde coloré (vert ou rouge)
- Montants formatés (ex. « 125 000 FCFA ») avec `Intl.NumberFormat('fr-FR')`, persistance dans `localStorage`

CONTRAINTES TECHNIQUES

- Pas de `alert()` : les erreurs s'affichent dans le DOM
- Montants stockés en nombres, jamais en chaînes
- Totaux toujours recalculés, jamais stockés
- Attributs HTML natifs (`required`, `min`, `type=date`) en plus du JavaScript

PISTES DE BONUS

- Plafond par catégorie avec alerte visuelle
- Tri des opérations par montant ou par date
- Export des opérations au format CSV

COMPÉTENCES TRANSVERSES

Rigueur sur les types de données (nombre vs chaîne), tests avec cas limites (montant négatif, vide, très grand), qualité des messages d'erreur.

5

PHASE B · STRUCTURE & DONNÉES

Squelette de l'application

Navigation, routage par hash et modules ES

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
4 à 6 h	●●●○○	Modules ES (`import` / `export`), événement `hashchange`, mise en page Grid + Flexbox	Coque LifeBoard : barre latérale + intégration des étapes 1 à 4

OBJECTIF PÉDAGOGIQUE

Passer de pages isolées à une vraie application : découper le code en modules, gérer la navigation sans rechargement et structurer un projet qui va grandir. Étape charnière du parcours.

DESCRIPTION FONCTIONNELLE

Une barre latérale permet de passer de l'Accueil aux Tâches puis au Budget sans recharger la page. L'URL (`#/taches`, `#/budget`) reflète la section affichée et le bouton « Précédent » du navigateur fonctionne.

FONCTIONNALITÉS CLÉS

- Mise en page avec barre latérale (Grid / Flexbox), menu repliable sur mobile
- Routeur minimaliste : lecture de `location.hash`, affichage de la section, page « introuvable »
- Chaque module exporte une fonction `init()` appelée à la première visite
- Module `storage.js` partagé pour toutes les lectures et écritures `localStorage`
- Lien actif mis en évidence dans le menu

CONTRAINTES TECHNIQUES

- `<script type="module">` ; servir le projet avec une extension type « Live Server » (les modules ne fonctionnent pas depuis `file://`)
- Pas de variable globale partagée entre modules
- Aucune librairie de routage

PISTES DE BONUS

- Mémoriser la dernière section visitée
- Transition douce entre sections (CSS)
- Raccourcis clavier de navigation

COMPÉTENCES TRANSVERSES

Lecture des erreurs de console liées aux modules, refactorisation, gestion des dépendances entre fichiers, patience : c'est l'étape la plus « architecturale ».

6

PHASE B · STRUCTURE & DONNÉES

Station météo

Appels d'API avec fetch et async/await

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
5 à 6 h	●●●○○	`async` / `await`, Promesses, lecture d'une API JSON, erreurs réseau, états de chargement	Module Météo

OBJECTIF PÉDAGOGIQUE

Maîtriser le flux complet d'un appel réseau : requête, attente, vérification, traitement, affichage et gestion des erreurs. Apprendre à lire la documentation d'une API.

DESCRIPTION FONCTIONNELLE

L'utilisatrice cherche une ville, la choisit dans une liste de résultats et obtient la météo actuelle et les prévisions des jours suivants. Les données viennent de l'API gratuite et sans clé Open-Meteo (géocodage + prévisions).

FONCTIONNALITÉS CLÉS

- Recherche d'une ville (API de géocodage) avec liste de résultats cliquables (ville, pays)
- Météo actuelle : température, vent, description avec icône (code météo → emoji)
- Prévisions sur 5 jours (minimum / maximum)
- États d'interface explicites : chargement, succès, erreur (ville introuvable, hors ligne)
- Dernière ville consultée mémorisée et rechargée au démarrage

CONTRAINTES TECHNIQUES

- `async` / `await` + `try…catch` obligatoires
- Vérifier `response.ok` avant `response.json()`
- Aucune clé d'API secrète dans le code
- Jamais d'`innerHTML` avec des données d'API
- Fonctions séparées : chercher, récupérer, afficher

PISTES DE BONUS

- Villes favorites
- Bascule °C / °F
- Géolocalisation (`navigator.geolocation`)

COMPÉTENCES TRANSVERSES

Lecture de documentation d'API, onglet « Réseau » des DevTools, résilience face à l'imprévu (coupure internet), esprit critique sur les données externes.

7

PHASE C · APPLICATIONS RICHES

Carnet de notes

CRUD complet, recherche instantanée et tags

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
5 à 7 h	●●●○○	CRUD complet, identifiants uniques, `<dialog>`, anti-rebond (debounce), `sort`, accents	Module Notes

OBJECTIF PÉDAGOGIQUE

Pratiquer les quatre opérations fondamentales (Créer, Lire, Modifier, Supprimer) sur des données structurées, et soigner l'expérience de recherche et de tri.

DESCRIPTION FONCTIONNELLE

Un carnet pour stocker idées, cours et mémos. Chaque note possède un titre, un contenu et des étiquettes ; la recherche filtre la liste pendant la frappe.

FONCTIONNALITÉS CLÉS

- Création et modification d'une note dans une fenêtre `<dialog>` ; suppression avec confirmation
- Identifiant unique par note (`crypto.randomUUID()`), dates de création et de modification
- Recherche instantanée (titre + contenu), insensible à la casse et aux accents
- Étiquettes multiples et filtre par étiquette
- Tri par date de modification ou par ordre alphabétique

CONTRAINTES TECHNIQUES

- Anti-rebond d'environ 300 ms écrit à la main avec `setTimeout`
- Modification via l'`id`, jamais via l'index du tableau
- Affichage avec `textContent`
- Aucune librairie de recherche

PISTES DE BONUS

- Épingler des notes en haut de liste
- Mise en surbrillance du terme recherché
- Compteur de mots et enregistrement automatique du brouillon

COMPÉTENCES TRANSVERSES

Modélisation de données (quels champs ? quels types ?), gestion de plusieurs états d'interface, sens du détail et de l'expérience utilisateur.

8

PHASE C · APPLICATIONS RICHES

Suivi d'habitudes

Calendrier mensuel, séries et logique de dates

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
6 à 8 h	●●●●○	Calcul de calendrier, algorithme de série (streak), fonctions pures, `console.assert`	Module Habitudes

OBJECTIF PÉDAGOGIQUE

Aborder une vraie logique algorithmique : générer une grille de calendrier, parcourir des dates et calculer des séries. Introduction à l'écriture de fonctions pures et testables.

DESCRIPTION FONCTIONNELLE

L'utilisatrice définit des habitudes (sport, lecture, code…) et coche chaque jour accompli sur une grille mensuelle. L'application calcule la série en cours, la meilleure série et le taux de réussite.

FONCTIONNALITÉS CLÉS

- Ajout et suppression d'habitudes
- Grille mensuelle (CSS Grid) générée dynamiquement : un jour = une case cochable
- Navigation mois précédent / suivant, jour courant mis en évidence
- Série en cours, meilleure série et taux de réussite du mois, par habitude
- Données de la forme `{ habitudeId: ["2026-10-01", "2026-10-02"] }`

CONTRAINTES TECHNIQUES

- Dates stockées en chaînes `AAAA-MM-JJ` (évite les pièges de fuseau horaire)
- Fonctions de calcul pures, sans accès au DOM
- Au moins 5 `console.assert` sur le calcul des séries
- Aucune librairie de calendrier

PISTES DE BONUS

- Objectifs hebdomadaires (ex. 3 fois par semaine)
- Vue « heatmap » de l'année façon GitHub
- Rappel via l'API `Notification`

COMPÉTENCES TRANSVERSES

Décomposition d'un problème complexe en petites fonctions, tests manuels méthodiques, persévérance face aux cas limites (février, années bissextiles, changement de mois).

9

PHASE C · APPLICATIONS RICHES

Tableau de statistiques

Graphiques dessinés à la main avec Canvas

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
6 à 8 h	●●●●○	API Canvas 2D, mise à l'échelle, agrégation avec `reduce`, `mousemove`, `requestAnimationFrame`	Module Statistiques

OBJECTIF PÉDAGOGIQUE

Transformer des données en visualisations. Comprendre l'agrégation, les systèmes de coordonnées et concevoir une fonction de dessin réutilisable.

DESCRIPTION FONCTIONNELLE

Une page qui rassemble les chiffres clés des modules précédents : dépenses par catégorie, évolution du solde, régularité des habitudes, tâches terminées. Les graphiques sont dessinés sans bibliothèque.

FONCTIONNALITÉS CLÉS

- Diagramme en barres des dépenses par catégorie du mois
- Courbe de l'évolution du solde sur plusieurs mois
- Jauge de progression pour les tâches terminées et les habitudes
- Info-bulle au survol (valeur exacte) via `mousemove`
- Fonction réutilisable `dessinerBarres(canvas, donnees, options)` et légende

CONTRAINTES TECHNIQUES

- Ni Chart.js ni D3 : le but est de comprendre ce qu'ils font
- Agrégation (`reduce`) dans un module séparé du dessin
- Prise en charge de `devicePixelRatio` pour des traits nets
- Gestion du cas « aucune donnée »

PISTES DE BONUS

- Animation d'apparition des barres
- Graphique en anneau (camembert)
- Export du graphique en PNG (`canvas.toDataURL()`)

COMPÉTENCES TRANSVERSES

Pensée mathématique (proportions, échelles), séparation entre données et affichage, lecture de documentation technique dense.

10

PHASE D · FINALISATION

Mise en production

Thèmes, sauvegarde, accessibilité et déploiement

DURÉE ESTIMÉE	DIFFICULTÉ	NOTION NOUVELLE	MODULE LIFEBOARD
8 à 10 h	●●●●●	Variables CSS et thèmes, `Blob` / `FileReader`, accessibilité (ARIA), validation d'import, déploiement	Finitions LifeBoard + publication

OBJECTIF PÉDAGOGIQUE

Transformer un projet d'apprentissage en produit fini : personnalisation, robustesse, accessibilité, documentation et mise en ligne. Étape de synthèse qui mobilise tous les acquis.

DESCRIPTION FONCTIONNELLE

LifeBoard devient une application complète et partageable : thème clair ou sombre, sauvegarde et restauration des données en un clic, navigation au clavier et hébergement public.

FONCTIONNALITÉS CLÉS

- Thème clair / sombre via variables CSS, respect de `prefers-color-scheme`, choix mémorisé
- Export de toutes les données en fichier JSON, import avec validation du format
- Accessibilité : labels, contrastes, navigation clavier, annonces `aria-live`
- Design responsive vérifié sur mobile, tablette et ordinateur
- README complet et déploiement sur GitHub Pages

CONTRAINTES TECHNIQUES

- Import refusé proprement si le fichier est invalide (aucune donnée existante écrasée)
- Aucun `innerHTML` avec des données utilisateur
- Score Lighthouse « Accessibilité » ≥ 90
- Zéro erreur dans la console

PISTES DE BONUS

- Application installable hors-ligne (PWA : manifeste + service worker)
- Page d'accueil résumant chaque module
- Mode impression pour le budget (`@media print`)

COMPÉTENCES TRANSVERSES

Autonomie, rigueur de livraison (checklist), documentation, utilisation de Git, ouverture au retour utilisateur (faire tester l'application par un proche).

Suivi de progression

Fiche personnelle de **Madefo Fortune** : à compléter au fil des étapes, avec Louis lors de chaque revue.

N°	Étape	Début	Fin	Validée	Date de revue
1	**Accueil vivant**			☐	
2	**Liste de tâches**			☐	
3	**Tâches persistantes**			☐	
4	**Gestionnaire de budget**			☐	
5	**Squelette de l'application**			☐	
6	**Station météo**			☐	
7	**Carnet de notes**			☐	
8	**Suivi d'habitudes**			☐	
9	**Tableau de statistiques**			☐	
10	**Mise en production**			☐	

Mes objectifs personnels

Apprenante : Madefo Fortune

Mentor : Louis

Signatures et date

Évaluer et célébrer

Grille d'observation à remplir avec Madefo à la fin de chaque étape (niveaux : à consolider · acquis · maîtrisé).

Critère	Ce que l'on observe	À consolider	Acquis	Maîtrisé
Fonctionnement	Toutes les fonctionnalités clés marchent, y compris avec des saisies inattendues.	☐	☐	☐
Qualité du code	Noms explicites, fonctions courtes, pas de duplication, commentaires utiles.	☐	☐	☐
Gestion des erreurs	Cas limites prévus ; messages clairs pour l'utilisatrice ; console propre.	☐	☐	☐
Interface et CSS	Lisible, cohérente, responsive, retours visuels sur les actions.	☐	☐	☐
Débogage	Utilise la console et les DevTools avant de demander de l'aide.	☐	☐	☐
Autonomie	Planifie, cherche dans la documentation, explique ses choix avec ses mots.	☐	☐	☐

Jalons à fêter

- **Fin de l'étape 3** : première application qui garde ses données. Un vrai cap franchi.
- **Fin de l'étape 5** : LifeBoard devient une application à plusieurs écrans.
- **Fin de l'étape 6** : première donnée venue d'Internet affichée à l'écran.
- **Fin de l'étape 10** : projet en ligne, avec une adresse à partager avec ses proches et ses futurs recruteurs.

Ressources de référence

MDN Web Docs	`developer.mozilla.org`	Documentation de référence HTML, CSS, JavaScript
javascript.info	`javascript.info`	Cours moderne et progressif, en français
Open-Meteo	`open-meteo.com`	Documentation de l'API météo (étape 6)
Can I use	`caniuse.com`	Compatibilité des fonctionnalités selon les navigateurs
DevTools	`Touche F12`	Console, éléments, réseau, application, Lighthouse

UN MOT DE LOUIS

Madefo, chaque étape terminée est une vraie victoire. L'essentiel n'est pas d'aller vite, mais de comprendre ce que l'on construit : c'est ainsi qu'on devient autonome. Je serai là pour chaque revue.

— Louis, mentor